Improved Parallel Processing of Massive De Bruijn Graph for Genome Assembly

Li Zeng[1], Jiefeng Cheng[1*], Jintao Meng[1,2,3], Bingqiang Wang[4], Shengzhong Feng[1]

[1*] Shenzhen Institutes of Advanced Technology, CAS, Shenzhen, 518055, PR. China
[2] Institute of Computing Technology, CAS, Beijing, 100190, PR.China
[3] Graduate University of Chinese Academy of Sciences, Beijing, 100049, China
{li.zeng, jf.cheng, jt.meng, sz.feng}@siat.ac.cn
[4] Beijing Genomics Institute, Shenzhen, 518083, China
wangbingqiang@genomics.cn

Abstract. De Bruijn graph is a vastly used technique for developing genome assembly software nowadays. The scale of this kind of graph can reach billions of vertices and edges which poses great challenges to the genome assembly task. It is of great importance to study scalable genome assembly algorithms in order to cope with this situation. Despite some recent works which begin to address the scalability problem with parallel assembly algorithms, massive De Bruijn graph processing is still very time consuming which needs optimized operations. In this paper, we aim to significantly improve the efficiency of massive De Bruijn graph processing. Specifically, the time consuming and memory intensive processing are the De Bruijn graph construction phase and the simplification phase. We observe that the existing list ranking approach repeatedly performs parallel global sorting over all De Bruijn graph vertices, which results in a huge amount of communications between computing nodes. Therefore, we propose to use depth-first traversal over the underlying De Bruijn graph once to achieve the same objective as the existing list ranking approach. The new method is fast, effective and can be executed in parallel. It has a computing complexity of O(g/p) and communication complexity of O(g), which is smaller than the existing list ranking approach, here g is the length of genome reference, p is the number of processors. Our experimental results using error-free data show that, when the number of processors scales from 8 to 128, our algorithm has a speedup of 10 times on processing simulated data of Yeast and C.elegans.

Keywords: Parallelized Graph Algorithm, De Brujin Graph, Genome Assembler

1 Introduction

Whole Genome sequencing is one of the most fundamental problems in Biology. Since 1977, when sanger sequencing method appeared, tens of thousands of genome sequence has been sequenced. Due to the diversity of species, there are still a large number of species that need to be sequenced. Moreover, modern medical research shows that most of the diseases and gene have connections. Genome sequencing results can lead to reveal the mystery of genetic variation, and is now widely used in genetic diagnosis, gene therapy and drug design, etc[4].

Whole Genome sequencing has two main steps. The first step is to obtain the sequence of short reads. It needs amplifying the DNA molecule (multiples of amplification is referred to as coverage) first, and then these DNA molecules are randomly broken into many small fragments (each small fragment is referred to as a read), the short read is then determined by sequencing devices, which is illustrated in Figure 1. The second step is to reconstruct genome sequence from these short reads, which is usually referred as *genome assembly*. Genome assembly problem is proved to be is NP-hard [1], reduced from the Shortest Common Superstring(SCS) problem.

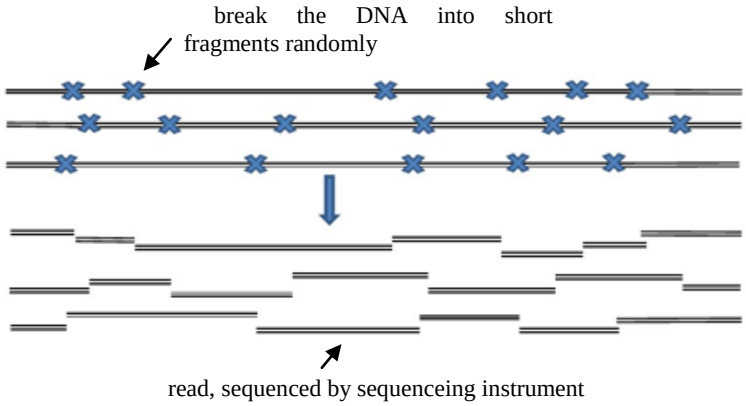

Figure 1 DNA and its reads

There are two types of sequence assembly algorithms. one is called overlap graph model [2]. On an overlap graph, each vertex corresponds to a read. There is a directed edge between two reads if overlap exists between them and the overlapping length exceeds a certain threshold. Therefore, sequence assembly problem is converted into finding a Hamilton path through each node in the overlap graph, which is NP-hard [2]. Another model is based on De Bruijn graph [3]. In a De Bruijn graph, each read is cut into small fragments of length k, called k-mer. Each k-mer contributes a node in the graph. If there are k-1 characters overlapping between two adjacent k-mers, then a directed edge exists between the two corresponding nodes. In this way, each read can produce a corresponding path in the graph. So the sequence assembly problem has been transformed into finding the shortest tour of the De Bruijn graph that include all the read path [4]. The problem is extremely difficult because repeat fragments of various lengths exist in the original sequence. Also, sequencing machines will

introduce errors into these reads, where the error rate is about 1%~3% in modern sequencing machines. These two issues make the sequence assembly more complicated.

Many optimization criteria and heuristics have been presented to deal with assembly problem. Early, the length of fragments obtained by the Sanger sequencing method can be achieved 200bp (base pairs). Overlap graph model algorithm is more effective, but the cost of sequencing is relatively high. For example, the Human Genome Project, by the completion of the first-generation sequencing technology, already spent $3 billion in 3 years. More recently, genome sequencing entered in a era of large-scale applications due to the second-generation sequencing technology (also known as the next-generation sequencing technology), such as Solexa, 454, SOLID [5]. There are three prominent features of the second-generation sequencing technology, namely, high throughput, short sequence and high coverage. For high-throughput sequencing, a sequencing machine can simultaneously sequence millions of reads, which greatly reduces the whole cost. For short sequence, it stands for that the sequence length is between 25-80 bp in general. In this regard, genome assembly has to greatly increase DNA coverage [5] to ensure the integrity of the information. With the increasing of the coverage, the total number of reads also increases greatly. For overlap graph , then the size of the graph will significantly increase accordingly. But for De Bruijn graph, the size of the graph is only linear to the length of the reference genome. Therefore, in the face of the second generation of gene sequencing technology, De Bruijn graph model has a great advantage and become much popular. The proposed algorithms based n De Bruijin graph includes Euler[3], ALLPATHS[6], Velvet[7], IDBA[8], SOAPdenovo[9], ABySS[10] and YAGA[11]. Euler, ALLPATHS, Velvet and IDBA algorithms are serial algorithms, only suitable for small data sets. SOAPdenovo is multi-threaded SMP mainframe-based algorithm, which can process a large data set. But it also spends more than 40 hours on human genome [9] and the required infrastructure is very expensive.

The main phases of genome assembly based on De Bruijn graph is as follows: First, De Bruijn graph construction. Second, errors correcition. To remove the known erroneous structures in the De Bruijn graph. Third, graph simplification. It is to compress a single chain in the De Bruijn graph into a single compact node, which also generates preliminary contig information. Fourth, generating scaffold. It is to merge contigs from the previous step with pair-end information obtained by sequencing, then to derive longer contigs (known as scaffold). Fifth, to ouput the obtained scaffold. Generally, the first phase has the largest memory consumption, and the second phase and the third phase has the largest time expense. Initially, The graph after constructed is extremely sparse, chain nodes ratio is more than 99% [12].

YAGA[11] is a distributed algorithm with well parallelism and scalability which represent state of the art. In this paper, we improve the parallel processing of De Bruijin graph by constructing a De Bruijn graph in distributed manner for the construction phase and then a parallel graph traversal to obtain all chains for the simplification phease. After assuming that the error has been removed, our work focuses on improving the efficiency of parallel graph simplification. The main contribution of this paper is to formulate the graph simplification problem into graph traversal problem. In YAGA algorithm, Graph simplification is approached as a list ranking problem with a total of 4 times global sorting and a multi-round recursion.

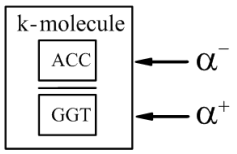

Figure 2 k-mer and k-molecule

Our algorithm directly traverses from end node to access all chains and visit each node only once. Our algorithm has a computing complexity of O(g/p) and communication complexity of O(*g*), which is smaller than YAGA algorithm, here *g* is the length of genome reference, *p* is the number of processors.
Section 2 gives the definition of De Bruijn and the detailed description on graph construction. Section 3 describes parallel graph simplification. We show experimental results in section 4. Section 5 concludes this paper.

2 Parallel De Bruijn Graph Construction

We first explain the definition of De Bruijin graph. Then, in order to achieve efficient parallel De Bruijin graph construction, we discuss a compact representation of De Bruijin graph. Finally, a parallel De Bruijin graph construction is given, which distributes the storage of the graph across multiple computational nodes.

2.1 Defining De Bruijn Graph

Let *s* be a DNA sequence of length *n*, which is a sequence consists of the four bases, namely *A*，C，T and *G*. A substring derived from *s* with length *k*, represented as $s[j]s[j+1]...s[j+k-1], n-k+1 \geq j \geq 0$, is called as a *k*-mer. The set of all k-mers derived from *s* is referred to as the k-spectrum of *s*. For a k-mer α, $\overline{\alpha}$ represents the reverse sequence of α，*s* complementary sequence (for example $\alpha = AAGTA$, then $\overline{\alpha} = TACTT$). $\overline{\alpha}$ can be obtained by first complementing each character in \alpha and then reversing and the resulted sequence. The complementary rules are $\sum: \{ A \to T,\ T \to A,\ C \to G,\ G \to C \}$.

A k-molecule represents a pair of k-mers $\{\alpha, \overline{\alpha}\}$, and α and $\overline{\alpha}$ is reverse complementary. The notation $>$ is the ordering relation between the two k-mers with length k, such that $\alpha > \overline{\alpha}$ indicates that α is larger than $\overline{\alpha}$ lexicographically. We designate the lexicographically larger one as the positive k-mer，denoted α^+, and the other one as negative k-mer，denoted α^- (as shown in Figure 2), here there is $\alpha^+ > \alpha^-$. When the context is clear, α^+ stands for a k-molecule $\{\alpha, \overline{\alpha}\}$, that is $\alpha^+ = \{\alpha^+, \alpha^-\} = \{\alpha, \overline{\alpha}\}$. Fig.2 depicts the relationship between a k-mer and its corresponding k-molecule with an example.

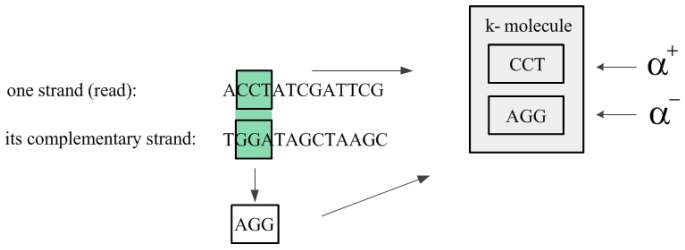

Figure 3 k-molecule from read

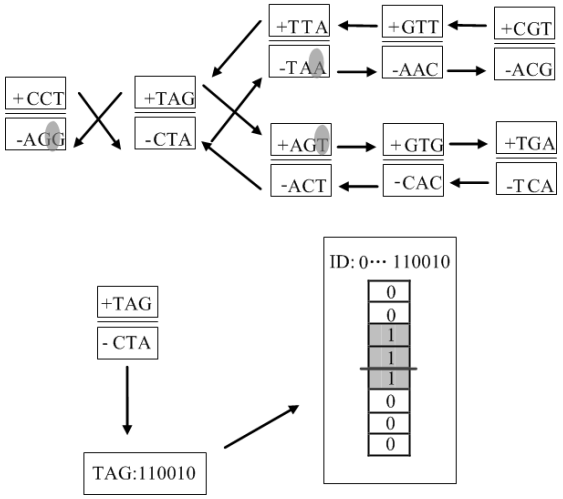

Figure 4 The structure of De Bruijin Graph node

2.2 Structure of De Bruijn Graph

The node structure in our De Bruijn graph is different from the existing one of YAGA algorithm [11]. Specifically, we design a compact representation of De Bruijin graph, where an ID field and an arc field are devised for every node. And this node structure not only uniquely identifies a node by the ID field, it also encodes all incident edges for that node in the arc field. Therefore, our De Bruijin graph representation does not store edges explicitly.

Let's consider a De Bruijin graph node, which is a k-molecule (as shown in figure 3). We compute an 64 bit unsigned integer based on the sequence of all characters in the positive k-mer using the rule {A: 00, C: 01 G: 10 T: 11} (shown in Figure 4). In turn, the 64 bit unsigned integer forms the ID field (or simply ID) of the node. Since we have a maximum value for k as 31, therefore, all characters in the

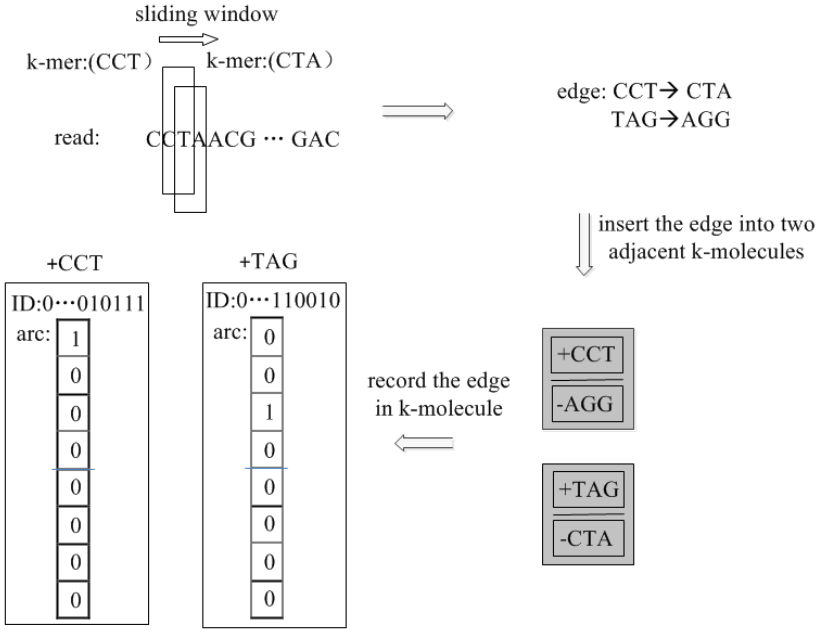

Figure 5 Obtaining All k-mers

positive k-mer can at most encode the low 2k bits of the ID. And the remaining high bits are filled with 0.

All edges are stored in the arc field of the node. Because DNA consists of four bases{A,C,G,T}, A node can connect to eight adjacent nodes at most, corresponding to eight edges. Since the two adjacent nodes have k-1 characters overlapped in their k-mers, positive or negative, only one different character can be found over the two adjacent nodes. So we can simply use this character to represent the corresponding edge. We use an 8-bit char type variable to store these edges. The bit with value of 1 indicates that the existence of this edge, otherwise, it means there is no corresponding edge. The rule is: the top 4 bits represent the edges of postive k-mer, 0 to 3 corresponding to the A C G T; the after 4 bits represent the edges of negative k-mer, 4 to 7 corresponding to A C G T. For example, the node TAG owns three edges connected to three nodes (TTA AGT and TAG), as defined above, we use the positive k-mer TAG to represent the node. We set the third bit of arc field to 1. It means that a edge exist from the positive k-mer(TAG) to the negative k-mer(AGG) of node CCT, for the number of the bit is the third and in the top 4. The edges record of node TAG is shown in Figure 4. The bits of the shaded part in Figure 4 with value of 1 represent the existence of these edges. Accordingly, only 9 bytes is used to store one node in our De Buijn graph.

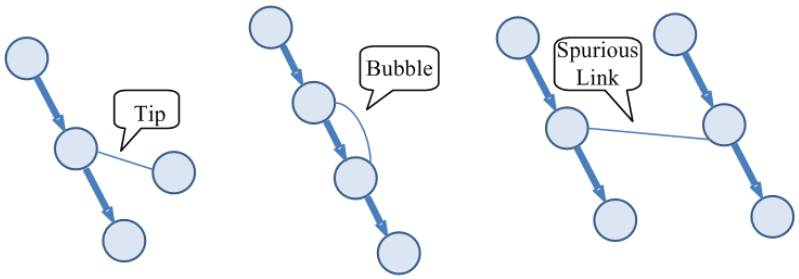

Figure 6 Three types of erroneous connections

2.3 Parallel De Bruijn Graph Construction Algorithm

The parallel De Bruijin graph construction algorithm have three main steps. The first step is to read all short sequences in parallel; The second step is to obtain all k-mers and the edges between any two consecutive k-mers, and to obtain all k-molecules based on those k-mers, repeated edges in a k-molecule are merged. The third step to distribute those k-molecules in a number of computational nodes. The details are given below.

Step 1: Read short sequences in parallel.

Each processor is assigned with different blocks of short sequences according to the rank of processor and the file size of short sequences. Assume that the length of the file is L and

there aren processors, then the i-th processor get short sequences from the $((i-1)*L)/n$ to $(i*L)/n$.

Step 2: Parallel segmentation of k-mers (see Figure 5).

Each processor cuts all short sequences into *k*-mers by a sliding window with length *k* from the first *k*-mer to the last *k*-mer. Then, each *k*-mer is handled as follows:

a) By the rules of complementation, we get all reverse complementary k-mers of the existing k-mers. Moreover, we obtain all k-molecule $\{\alpha^+ \geq \alpha^-\}$ based on those k-mers. The lexicographically larger k-mer is encoded by the rule {A: 00, C: 01 G: 10 T: 11} as the ID of the k-molecule.

b) Record edges between adjacent k-molecules . Form the compact De Bruijin graph node representation for all k-molecules.

Step 3: Distribute all k-molecules.

Each processor stores a partition of the De Bruijin graph. All processors are assigned with a certain portion of De Bruijin graph nodes with a hash function over the 64-bit node representation. Specifically, the location of a De Bruijin graph is computed by taking the ID of the k-molecule mod of the number of processors. So, all k-molecule are mapped to different processors by an all-to-all communication operation. After this step, any processor can computed an node

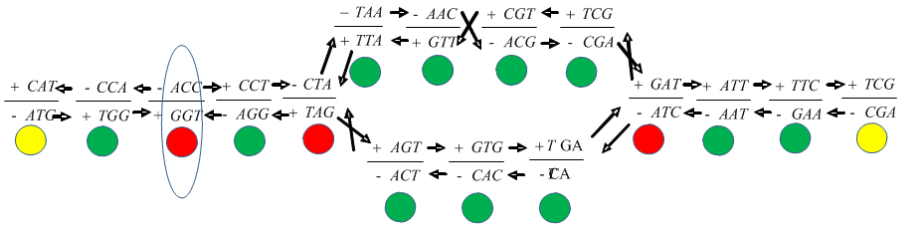

Figure 7 De Bruijn graph in our algorithm

directly by the same hash function. This is to boost the performance of the following graph simplification phase.

3 Parallel Simplification of De Bruijn Graph

In this section, we first explain the problem of De Bruijin simplification. Then, we discuss our parallel De Bruijin graph simplification algorithm based on concurrent graph traversal starting from multiple sources in the graph.

3.1 The De Bruijin Graph Simplification Problem

There can be a lot of sequencing errors introduced by gene sequencing devices. Therefore, the De Bruijn graph obtained by the construction algorithm can have a large number of erroneous paths, which give rise to three types of *erroneous connections* in the graph, namely, *tip*, *bubble* and *spurious link*, which are depicted in Figure 6. Therefore, after the De Bruijn graph construction, the immediate processing is to remove those erroneous connections using the knowledge from Biology study [12]. After this, the De Bruijn graph is error-free and very sparse such that it contains a large number of chains (as shown in Figure 7) [13]. Therefore, an important processing is to simplify the De Bruijn graph by finding all chains in the graph. After simplification, the size of the graph can be reduced sharply, almost about 90% off and the graph can be processed for latter tasks such as processing branches and repeats in a single machine easily.

Since our focus in this paper is an efficient and effective parallel De Bruijn graph simplification algorithm, we assume the processing to remove erroneous connections is completed. So, in the following of this paper, our discussion is on the De Bruijn graph without erroneous connections, that is, the error-free data. First, we give a few notations and a detailed description of the problem. We call a node with a degree of 1 as the *1 degree node*. A node with degree of 2, by an incoming edge and a outgoing edge, is called as the *chain node*. The chain node can lead to a chain. all nodes with a degree of 2, by two incoming edges or two outgoing edges, asl well as all nodes with the degree larger than 3 is *bifurcation node*. A *chain* is a simple path in

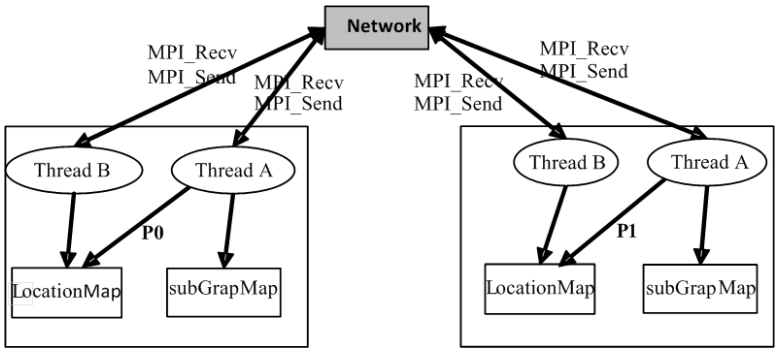

Figure 8 Thread B mergeing and communication thread

the graph containing chain nodes only. Nodes at both ends of the chain are called as *end node*. Although an end node is not part of the chain, the chain can be obtained by traversal from it.

In figure 7, the green nodes are chain nodes, which are the target of simplification. The red node is bifurcation node and the yellow is 1 degree node, both of them are end node. Although the degree of the circled node in figure 7 is 2, it cannot be traversed for it has two incoming edges or two outgoing edges.

3.2 Parallel De Bruijin Graph Simplification Algorithm

Overview. Our algorithm directly traverses from an end node of a chain to the other end node to simplify the chain.

Therefore, we use two threads for each processor, which collectively performs the concurrent graph traversal to identify all chains. As shown in Figure 8, thread A is responsible for the simplification task as to merge all chain nodes identified over a chain, on the other hand, thread B is dedicated for inter-process communications. The communication is for resolving local requests and dealing with other processor's requests on traversing end nodes.

Data structure. Each processor has two Maps. One is locationMap, which store nodes before simplification. Another is subGraphMap which stores nodes after simplification. During initialization, the subGraphMap is empty and the subGraphMap is inserted with nodes during the simplification by the process of identifying chains.

Algorithm. Based on the above data structure, we outline the algorithm as below:

Step 1: Initialization: De Bruijn graph is distributed on all processor and each processor has a locationMap and a subGraphMap.

Step 2: Traversal of the graph: Every processor starting to traverse from an end node in local locationMap. Since a chain has two ends nodes, traversal can be divided into the following three cases: 1) a processor visits a same chain twice from the two end nodes; 2) two different processors visit the same chain twice, each one from a different end node; 3) two processors visit the same chain simultaneously

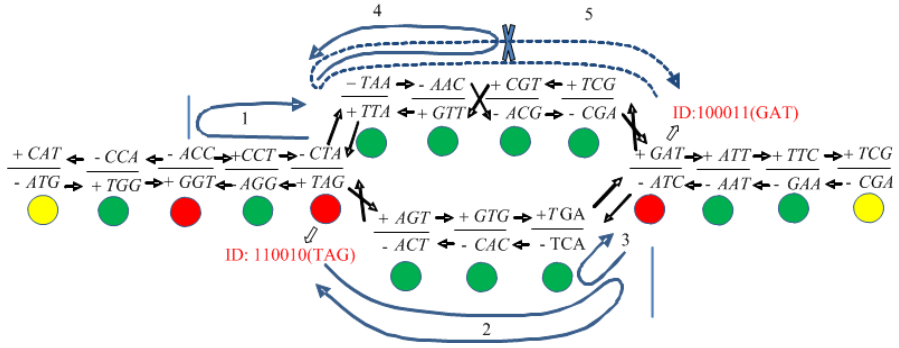

Figure 9 The procedure of getting chains

by starting from the two end node at almost the same time. In order to reduce repeated access to the same chain, we add a endNodeID variable to store the ID of end node in the direction of the end. According to the ID number, the merging thread can determine whether to continue or exit.

Step 3: Simplification: Delete all chain nodes and append the their information to the corresponding end node, and then insert them into subGraphMap.

Step 4: Select another end node to repeat the processing from Step 2 until no end node can be found.

3.3 Analysis

YAGA algorithm transforms the problem of simplifying all chains into list ranking problem which requires four times of global sorting over all graph nodes. In this procedure, a large number of nodes are moved four times. The time and communication cost is very large. The computational complexity of four times global sorting is $O((g/p)\ln(g/p))$, and the communication complexity is $O(g)$. In addition to the cost of four times of global sort, the cost of other involved processing is also very large.

In our algorithm, each processor directly traverses from the local end node to get all chains. A chain only holds two end nodes, so each chain node is accessed twice at most. Also our algorithm successfully prevents a chain from being visited twice. So, In the worst case, all nodes are calculated twice, the time complexity is $O(g)$. And each node is requested twice at most, the communication complexity $O(g)$.

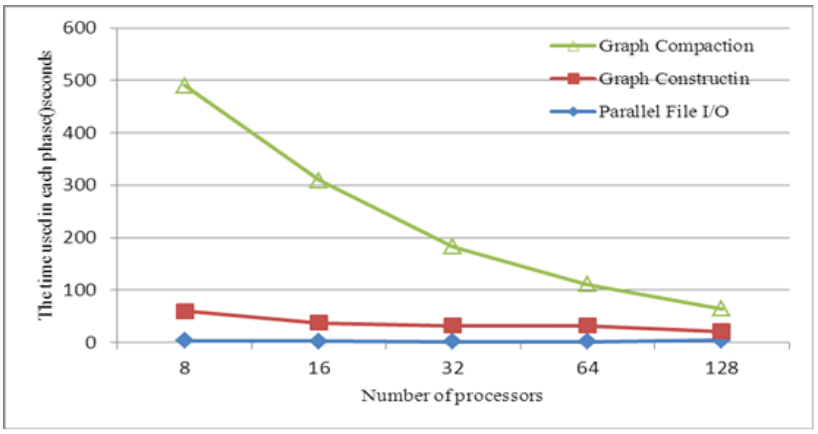

Figure 10 The time used in three phases on Yeast dataset

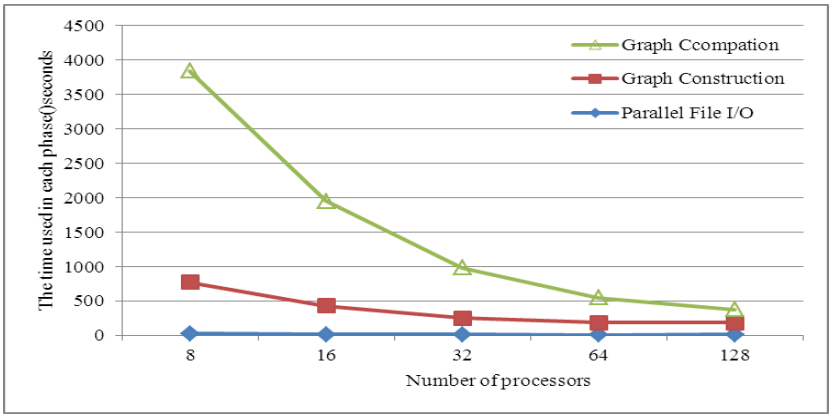

Figure 11 The time used in three phases on C.elegans dataset

4 Performance Evaluation

Our assembly software is written in C++ and MPI. The experimental platforms includes 10 servers interconnected by InfiniBand network, which is configured as 16-core, 32G shared memory. We choose Yeast and C.elegans genomes, using Perl scripts automatically generated yeast test data with 17, 007, 362 reads and c.elegans test data, which consists of 140, 396, 108 reads. The length of reads ranges from 36bp to 50bp with error rate of 0, and the coveage for both data is 50X Our goal is to demonstrate the scalability of our assembler to graph construction and graph compaction on a large distributed De Bruijn graph.

Firstly, we tested assembler on the Yeast data set. The runtime is dispalyed in figure 10 and the time is divided into three phrases. First, parallel read file from a distributed file system. Second, graph construction. Third, graph compaction. The most part time on the third phase is network transmission cost. But, in our algorithms most nodes only be moved once, that is an efficient optimization. Figure 10 indicates that the algorithm has good scalability. When the number of processors is increased from 8 to 128, the total time is reduced from 490s to 63s that meas the speedup is about 8 times. The C.elegans dataset is 10 times larger than Yeast dataset. The running time shown in Figure 11. When the number of processors is increased from 8 to 128, the total time is reduced from 3844s to 375s with the speedup being 10 times.

4 Conclusion

We propose to use depth-first traversal over the underlying De Bruijn graph once to simplify it. The new method is fast, effective and can be executed in parallel. By testing the two data sets, the experimental results show that the algorithm has great scalability. After the completion of the graph simplification, the scale of the figure is sharply reduced, thus the graph simplification is of great value. The latter stage includes resolving branches and repeats using pair-end information in a single machine. We will study these problems in our future work.

References:

[1] KUNDETI V K, RAJASEKARAN S, DINH H, et al. Efficient parallel and out of core algorithms for constructing large bi-directed de Bruijn graphs [J]. Bmc Bioinformatics, 2010, 11(560).

[2] KECECIOGLU J D, MYERS E W. Combinatorial algorithms for DNA sequence assembly [J]. Algorithmica, 1995, 13(1): 7-51.

[3] PEVZNER P A, TANG H, WATERMAN M S. An Eulerian path approach to DNA fragment assembly [J]. Proceedings of the National Academy of Sciences of the United States of America, 2001, 98(17): 9748-53.

[4] MEDVEDEV P, GEORGIOU K, MYERS G, et al. Computability of models for sequence assembly [J]. Algorithms in Bioinformatics, 2007, 289-301.

[5] JACKSON B G, ALURU S. Parallel Construction of Bidirected String Graphs for Genome Assembly [J]. 2008, 346-53.

[6] BUTLER J, MACCALLUM I, KLEBER M, et al. ALLPATHS: de novo assembly of whole-genome shotgun microreads [J]. Genome Res, 2008, 18(5): 810-20.

[7] ZERBINO D R, BIRNEY E. Velvet: algorithms for de novo short read assembly using de Bruijn graphs [J]. Genome Res, 2008, 18(5): 821-9.

[8] PENG Y, LEUNG H, YIU S, et al. IDBA–A Practical Iterative de Bruijn Graph De Novo Assembler, F, 2010 [C]. Springer.

[9] LI R, ZHU H, RUAN J, et al. De novo assembly of human genomes with massively parallel short read sequencing [J]. Genome Res, 2010, 20(2): 265-72.

[10] SIMPSON J T, WONG K, JACKMAN S D, et al. ABySS: a parallel assembler for short read sequence data [J]. Genome Res, 2009, 19(6): 1117-23.

[11] JACKSON B, REGENNITTER M, YANG X, et al. Parallel de novo assembly of large genomes from high-throughput short reads, F, 2010 [C]. IEEE.